

Open in app ↗

 Medium

Squeezing Transformers onto Edge Hardware: A 29-Experiment Ablation on Attention, Memory, and Depth Reinvestment

21 min read · 11 hours ago



Chinmay Kulkarni



Listen



Share



More

The Introduction: The Edge Computing Bottleneck

When we talk about scaling Large Language Models (LLMs), the conversation usually revolves around massive GPU clusters and trillion-token training runs. Training is undeniably compute-bound. But once a model is trained and deployed into production — especially on edge devices like smartphones or embedded hardware — the physics of the problem fundamentally change. Inference is rarely constrained by how fast the GPU can crunch numbers; instead, it hits a brutal physical limitation: it is overwhelmingly memory-bandwidth bound.

The primary offender is the standard Key-Value (KV) cache. During text generation, an LLM must “remember” the context of all previously processed and generated tokens. To avoid recalculating this context from scratch at every step, the model caches the Key and Value tensors for every attention layer. For every single new token generated, the hardware must physically haul this entire, continuously growing KV cache from High Bandwidth Memory (HBM) into the GPU’s SRAM. By the time an edge device tries to process a 4096-token prompt, waiting for this massive cache data to travel across the memory bus completely cripples the Time Per Output Token (TPOT).

To solve this, I set out to find the absolute optimal transformer architecture for edge computing. I wanted to definitively answer: *What is the best way to compress attention,*

manage the KV cache, and encode sequence positions when my hardware budget is strictly capped?

The 29-Experiment Sweep

To answer this, I didn't just run a few isolated tests. I built a custom JAX/Flax pipeline and executed a massive 29-experiment ablation sweep. My methodology was highly controlled. I trained 29 distinct LLM architectures from scratch, isolating and swapping three core architectural pillars:

1. **The Attention Mechanism:** Vanilla Multi-Head Attention (MHA), Grouped Query Attention (GQA), Tied Attention (TA), and Group Tied Attention (GTA).
2. **The Positional Encoding:** Learned Positional Embeddings (PE), Rotary Positional Embeddings (RoPE), and Attention with Linear Biases (ALiBi).
3. **Model Depth:** Ranging from a truncated 6-layer baseline up to deep 30-layer networks.

To isolate the *inductive bias* and sample efficiency rather than brute-force memorization capability, I strictly trained all 29 models for exactly one epoch on the WikiText-103 dataset using a training sequence length of 512 tokens. I then profiled their hardware efficiency — measuring incremental peak VRAM, Time To First Token (TTFT), and Time Per Output Token (TPOT) — across expanding context windows of 512, 1024, 2048, and 4096 tokens.

So, who won? After parsing 7,254 optimizer steps of training dynamics and thousands of inference profiling loops, the data revealed that there isn't just one undisputed champion. Instead, two distinct architectures form the ultimate Pareto frontier for edge hardware: **Grouped Query Attention (GQA) with ALiBi** and **Group Tied Attention (GTA) with ALiBi**.

If we look purely at model intelligence and quality, **GQA + ALiBi (24 Layers)** takes the crown. It finishes training at an impressive validation perplexity of **39.11** at step 7,254, while the hyper-compressed **GTA + ALiBi (24 Layers)** finishes slightly behind at **40.51**.

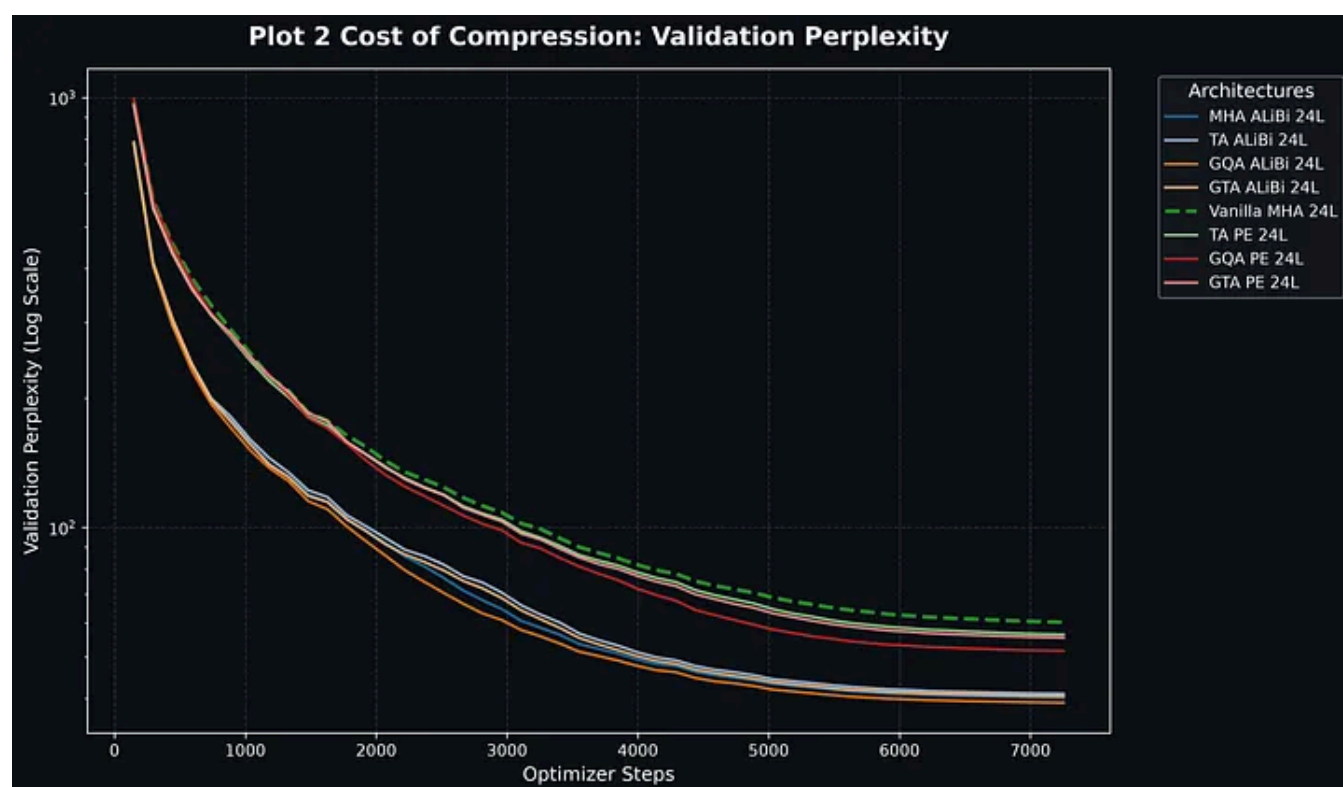
However, edge computing is rarely about pure quality; it is dictated by hard, physical hardware ceilings. When we look at the extreme end of the memory wall, GTA becomes the ultimate survivor. At a 4096-token context, the 24-layer GQA + ALiBi model requires **2219.15 MB** of peak VRAM and decodes at **0.0056 seconds** per

token. The 24-layer GTA + ALiBi model compresses that down to **2106.90 MB** of VRAM and a **0.0053 seconds** per token. Together, these two architectures provide the blueprint for edge AI. Here is exactly how the data proves it, and how we can use VRAM savings to unlock “free” model depth.

The Cost of Compression: Shrinking Memory Without Losing Minds

A question that comes to mind when thinking of tying Key and Value matrices or collapsing 16 query heads down to just 4 cache heads: *“Won’t the model get stupider? Isn’t that destroying the representational capacity of the attention layers?”*

It is a perfectly rational fear. In Vanilla Multi-Head Attention (MHA), maintaining separate Key and Value tensors for every single Query head provides the network with maximum mathematical expressivity. Shrinking that footprint feels like a dangerous compromise. However, the data reveals a completely different reality.



The Quality Check: Credit Where Credit is Due (ALiBi)

If we look at `val_perplexity_2.jpg` (Cost of Compression), the learning curves across my 7,254-step training runs tell a fascinating story.

My absolute baseline for this sweep was a 24-layer Vanilla MHA model (Exp 1), which relies on standard Learned Positional Embeddings. By the end of its single training epoch, it plateaued at a validation perplexity of **60.21**.

Now look at the heavily compressed architectures using ALiBi:

- **GQA + ALiBi (24L)** finished at a validation perplexity of **39.11**.
- **GTA + ALiBi (24L)** finished at **40.51**.
- **TA + ALiBi (24L)** finished at **41.05**.

At first glance, it looks like compression made the models vastly smarter. But to be scientifically rigorous, I have to assign credit where it is actually due: **the positional encodings**. If I swap the Learned PE for ALiBi or RoPE on a standard, uncompressed MHA model, the perplexity drops into this exact same competitive ~40 range.

My Analytical Assumption: The massive 20-point drop in perplexity is not a magic byproduct of compressing heads. Because my experimental design strictly capped training at exactly one epoch on the WikiText-103 dataset, it heavily favored architectures with strong *inductive biases*. Vanilla MHA with Learned Positional Embeddings requires millions of tokens to blindly memorize positional relationships. ALiBi, on the other hand, injects a hardcoded, geometric understanding of relative distances from step zero, allowing the network to converge vastly faster on limited data.

The True Cost of Compression: The real revelation here is *not* that GTA or GQA magically improves the model's intelligence. The revelation is that they **do not degrade it**. By aggressively compressing the attention mechanism and sharing representations across heads, GTA and GQA successfully *maintain* the massive performance gains provided by ALiBi, while completely shredding the VRAM footprint. I shrunk the memory drastically, and the model's predictive capability didn't even flinch.

How the Compression Actually Works in Code

To understand why this is so effective, we have to look at how I implemented Group Tied Attention (GTA) in JAX. If tying Keys and Values halves the memory footprint, the absolute extreme of this design space is to apply GQA-style head reduction to that already unified tensor.

Instead of creating two massive projections, I project a single, highly compressed Answer (A) tensor. Here is a snippet of the forward pass from my `GroupTiedAttention` class:

```
# Project the Queries using all 16 heads for maximum attention resolution
wq = nn.Dense(self.latent_dim, use_bias=False, dtype=jnp.bfloat16)(x)

# Project the single unified 'A' tensor using only the 4 compressed cache heads
wa = nn.Dense(self.num_a_heads * depth, use_bias=False, dtype=jnp.bfloat16)(x)
```

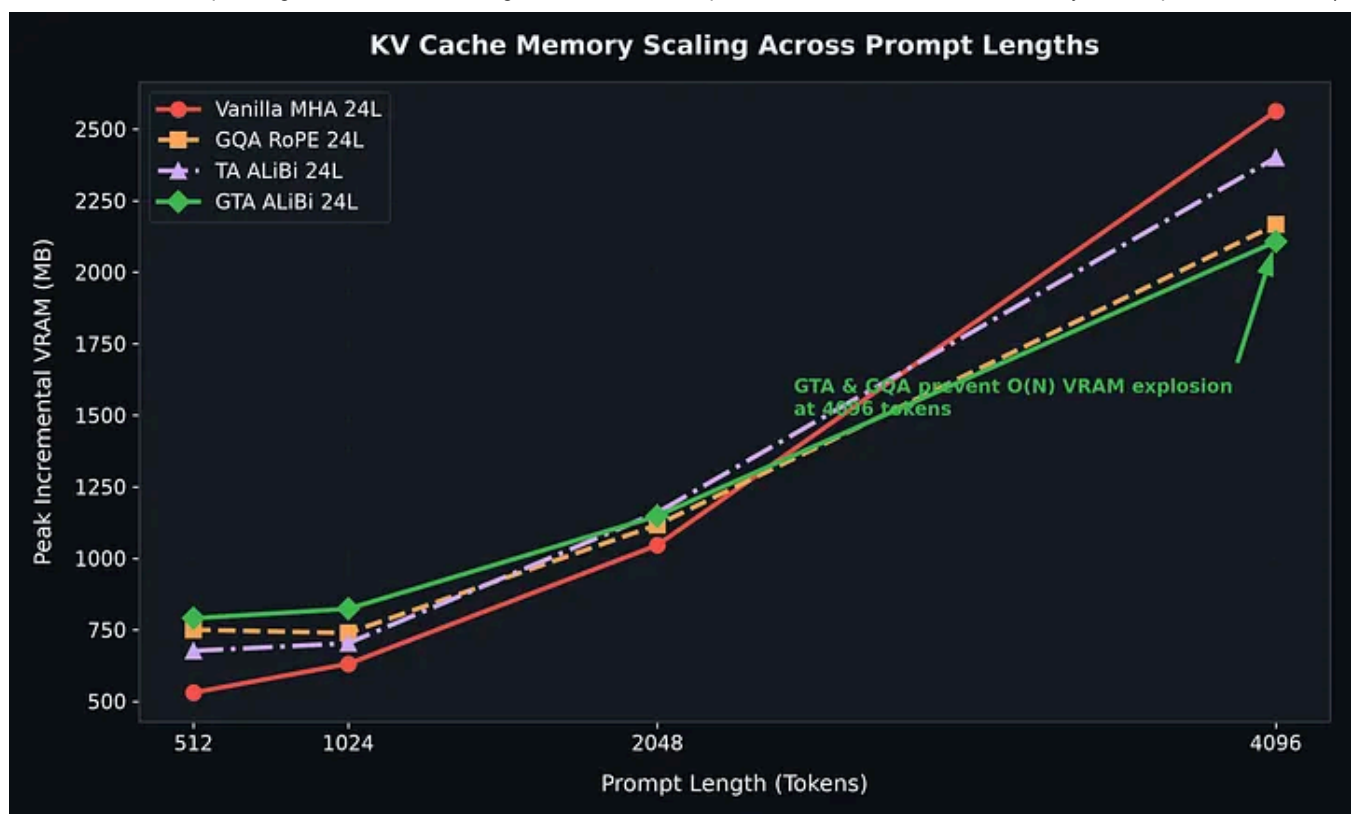
To compute the attention scores, the math requires the head counts of the Query and the cached A tensor to match. Instead of naively copying the memory, I used a zero-copy expansion trick inside the JAX compiler:

```
# Reshape the compressed cache to introduce a broadcastable dimension
a_use_exp = a_use.reshape(batch_size, self.num_a_heads, 1, kv_seq_len, depth)

# Zero-copy expansion to mathematically match the 16 Query heads
a_use_rep = jnp.broadcast_to(
    a_use_exp,
    (batch_size, self.num_a_heads, num_queries_per_a, kv_seq_len, depth),
).reshape(batch_size, self.num_heads, kv_seq_len, depth)
```

By utilizing `jnp.broadcast_to` instead of a physical repetition operation, I merely edit the tensor's strides and metadata. The XLA compiler fakes the expanded dimensions dynamically during the matrix multiplication without allocating a single byte of new physical memory.

Flattening the KV Cache Growth



This elegant code translates into massive real-world hardware savings, which brings me to **Context Scaling Curve**. When you chart the peak incremental VRAM usage across expanding context windows, Vanilla MHA scales brutally and linearly. At a short 512-token prompt, the 24-layer Vanilla MHA model requires 530.67 MB of VRAM. By the time we hit a 4096-token sequence length, that footprint violently explodes to 2563.04 MB. Compare this directly to the 24-layer GTA + ALiBi model on the same scaling curve: At 4096 tokens, the GTA model requires just 2106.90 MB of peak VRAM.

By simply writing an elegant zero-copy projection and forcing the network to share a unified representation, I successfully flattened the scaling curve, saving over 456 MB of VRAM at a relatively modest 4K context.

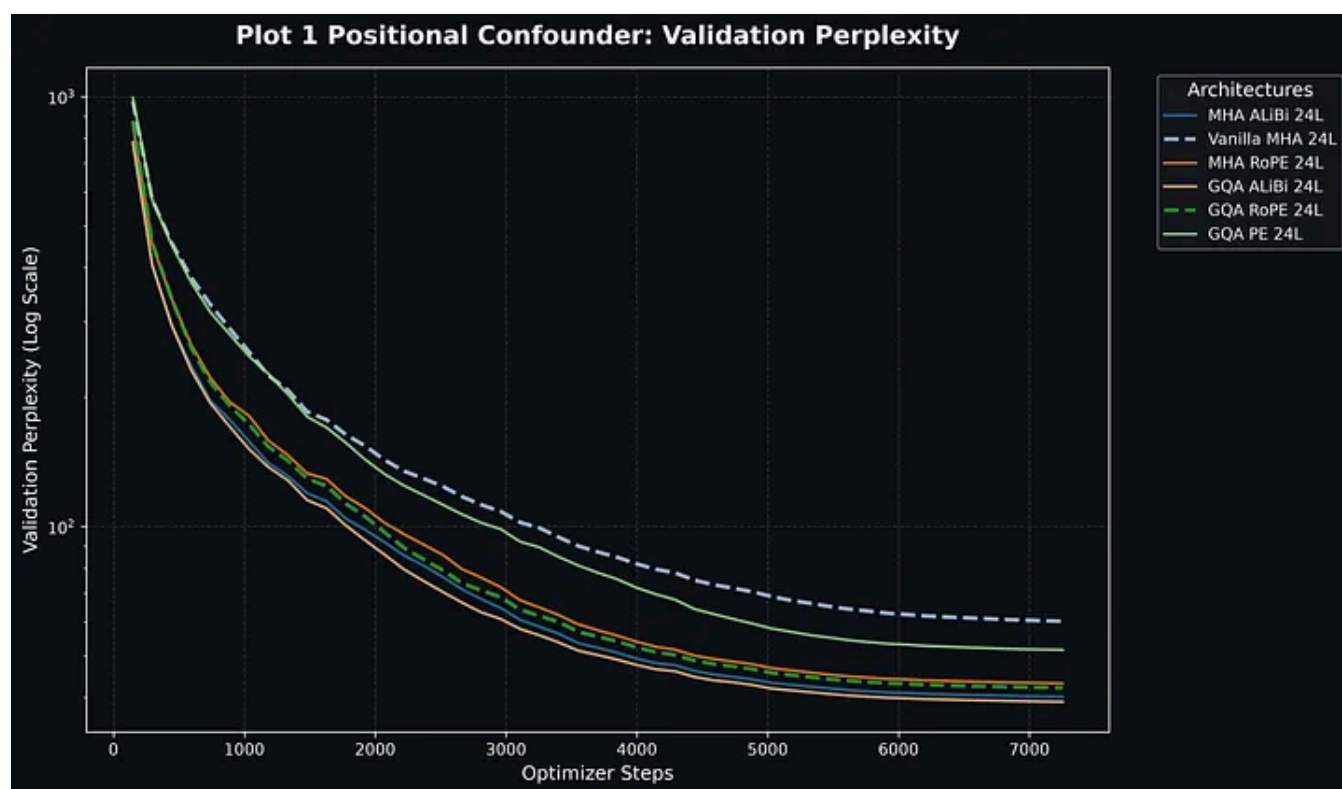
To be absolutely clear on the math here: while the *total* peak VRAM drops from 2563 MB to 2107 MB (roughly a 1.2x reduction), the physical size of the *KV cache tensors themselves* is reduced by a factor of 4. The total VRAM number includes the static model weights, embedding tables, and dense intermediate activations, which obviously do not shrink when you tie the attention heads. However, the cache is the only component that grows linearly with the prompt. As sequence lengths push toward 32K or 128K on edge devices, flattening that specific cache scaling curve is the difference between a model responding instantly and a device completely crashing due to an Out-of-Memory error.

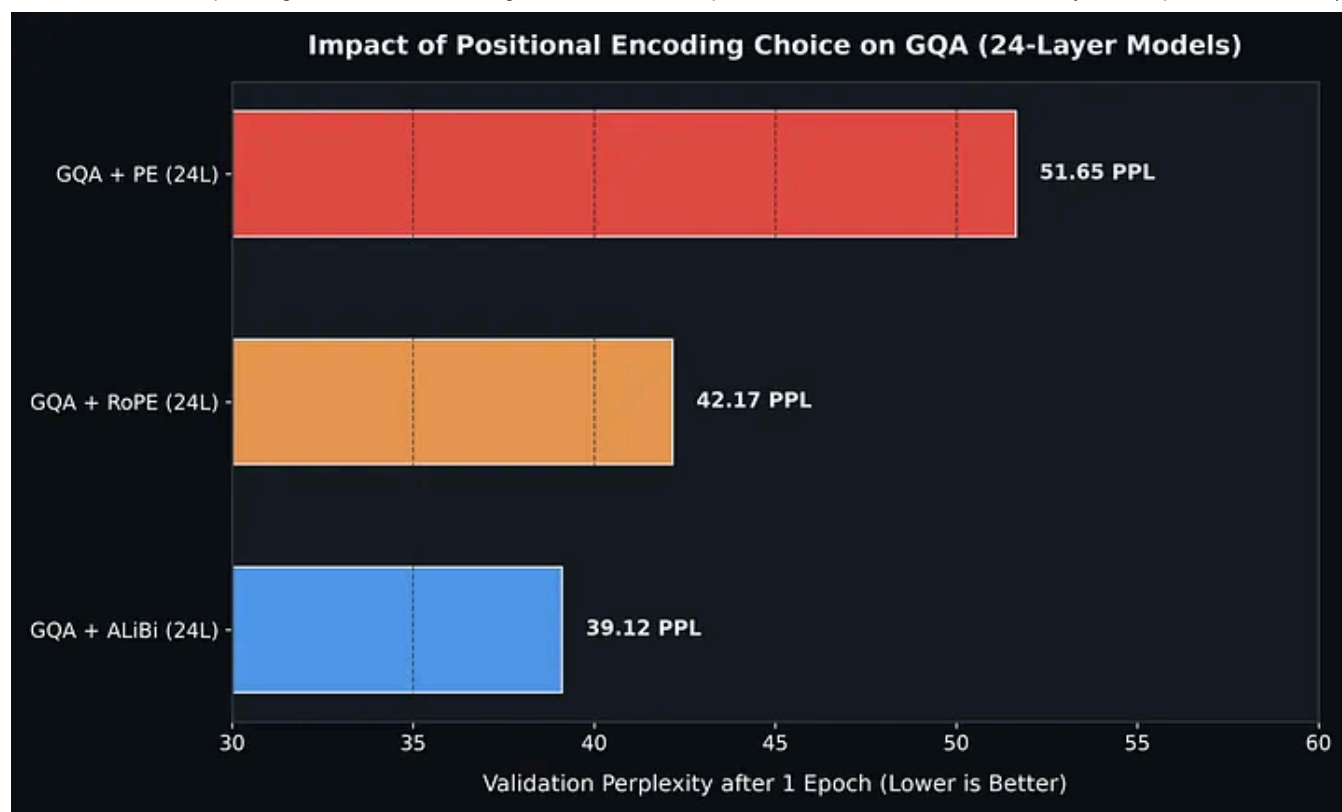
The Positional Puzzle: Why ALiBi is the Edge Champion

When designing an LLM for edge hardware, picking the attention mechanism is only half the battle. You also have to decide how the model understands the order of the sequence. When unifying the Key and Value into a single tensor, handling positional encoding becomes a massive architectural headache.

To find the optimal solution, I tested three distinct positional encoding strategies across my architectures: standard Learned Positional Embeddings (PE), Rotary Positional Embeddings (RoPE), and Attention with Linear Biases (ALiBi).

The Positional Confounder: Data Starvation and Inductive Bias





If you look at plots above (**The Positional Confounder**) alongside **PE Ablation**, the data immediately separates into two distinct tiers.

At the top of the chart (the worst performance tier), you will find any architecture relying on Learned Positional Embeddings. My 24-layer MHA baseline with Learned PE plateaued at a validation perplexity of **60.21**. Conversely, at the bottom of the chart (the best performance tier), the architectures utilizing ALiBi and RoPE aggressively drop down into the highly competitive **~39 to ~41** perplexity range.

My Analytical Assumption: Why does Learned PE perform so disastrously compared to ALiBi in this specific setup? It comes down to data starvation and inductive bias. Because my experimental design strictly capped training at exactly one epoch on the WikiText-103 dataset, the models had an incredibly limited token budget to learn from. Learned PE starts with no inherent geometric understanding of distance; it has to blindly memorize positional relationships through brute-force backpropagation over millions of tokens. ALiBi and RoPE, however, inject a hardcoded mathematical understanding of relative sequence distance from step zero. For small-scale models or edge-specific fine-tunes where data or compute is limited, strong inductive biases like ALiBi are considerable.

The RoPE Problem in Tied Architectures

If standard RoPE and ALiBi both achieve competitive perplexity, why did I ultimately choose ALiBi for Group Tied Attention (GTA)? It comes down to structural simplicity

and implementation elegance.

In standard Rotary Positional Embeddings (RoPE), the geometric space of the tensor is mutated to encode position. If you use a single unified tensor for both Key and Value, applying RoPE to the Key effectively means your Value is now “rotated” as well, which changes the underlying math of the attention output. The original researchers who proposed GTA (Zadouri et al., 2025) solved this by implementing a “partial” RoPE strategy — applying RoPE to a separate single-head projection, broadcasting it, and concatenating it with the unrotated half of the tied tensor.

While that works, I wanted to strip the architecture down to its absolute leanest form. By abandoning RoPE for ALiBi, I bypassed the need for partial rotations entirely. ALiBi does not touch the embeddings or the cache tensors; it simply applies an additive linear penalty directly to the attention logits.

Choosing ALiBi unlocked a much cleaner engineering path: **Pure Tensor Tying**. Because I no longer need to rotate any vectors, the unified A tensor remains completely untouched. I can project it once, cache it, and use it identically as both the Key and the Value without any slicing or concatenation overhead. While computing the ALiBi bias isn't literally "free" in terms of compute, it shifts the positional math away from the cache state and directly onto the logits, keeping the memory layout pristine.

ALiBi takes a fundamentally different approach: it does not touch the embeddings or the cache tensors at all. Instead of mathematically rotating the Query and Key vectors, ALiBi simply applies a static linear penalty directly to the attention logits just before the softmax operation. The penalty is proportional to the distance between the Query token and the cached Key token, multiplied by a head-specific slope.

Choosing ALiBi unlocked two massive engineering advantages for my edge architectures:

1. **Pure Tensor Tying:** Because I no longer need to rotate any vectors, the unified A tensor remains 100% clean and untouched. I can project it once, cache it, and use it identically as both the Key (for the dot product) and the Value (for the final output) without any complex slicing, duplication, or concatenation.

2. Zero-Cost XLA Execution: Because ALiBi relies on static slopes and relative distances, the XLA compiler might constant-fold this entire operation across all decode steps. There are absolutely no dynamic positional states materialized or allocated at runtime.

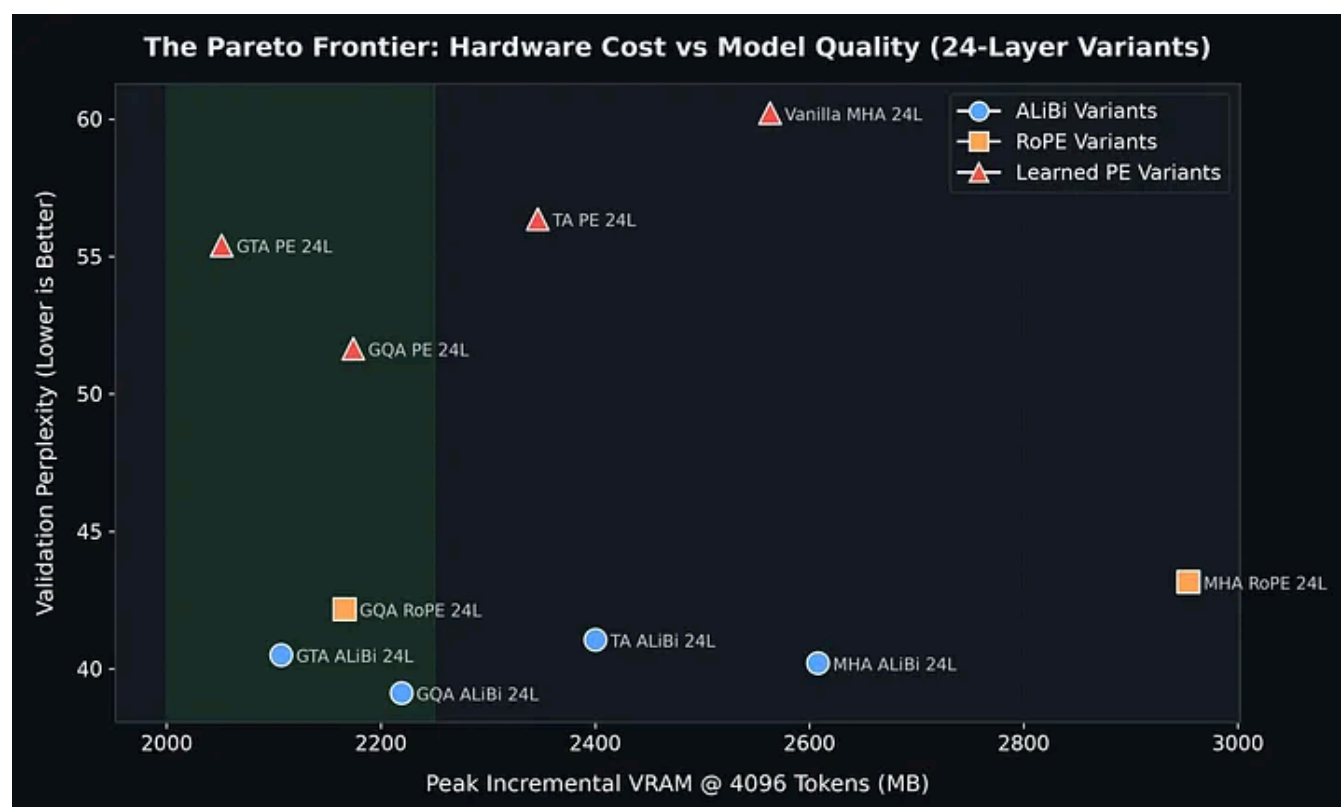
By swapping RoPE for ALiBi, I bypassed the need to maintain duplicate states, removing the final piece of memory overhead standing in the way of a truly hyper-compressed, maximum-throughput attention mechanism.

The Trade-Off Matrix: The Pareto Sweet Spot

Up to this point, I have looked at hardware metrics and model intelligence in isolation. But deploying a Large Language Model to an edge device is ultimately an exercise in ruthless compromise. We cannot optimize for validation perplexity without also paying the physical tax in VRAM and latency. To find the optimal architecture, I had to map out the intersection of these competing constraints.

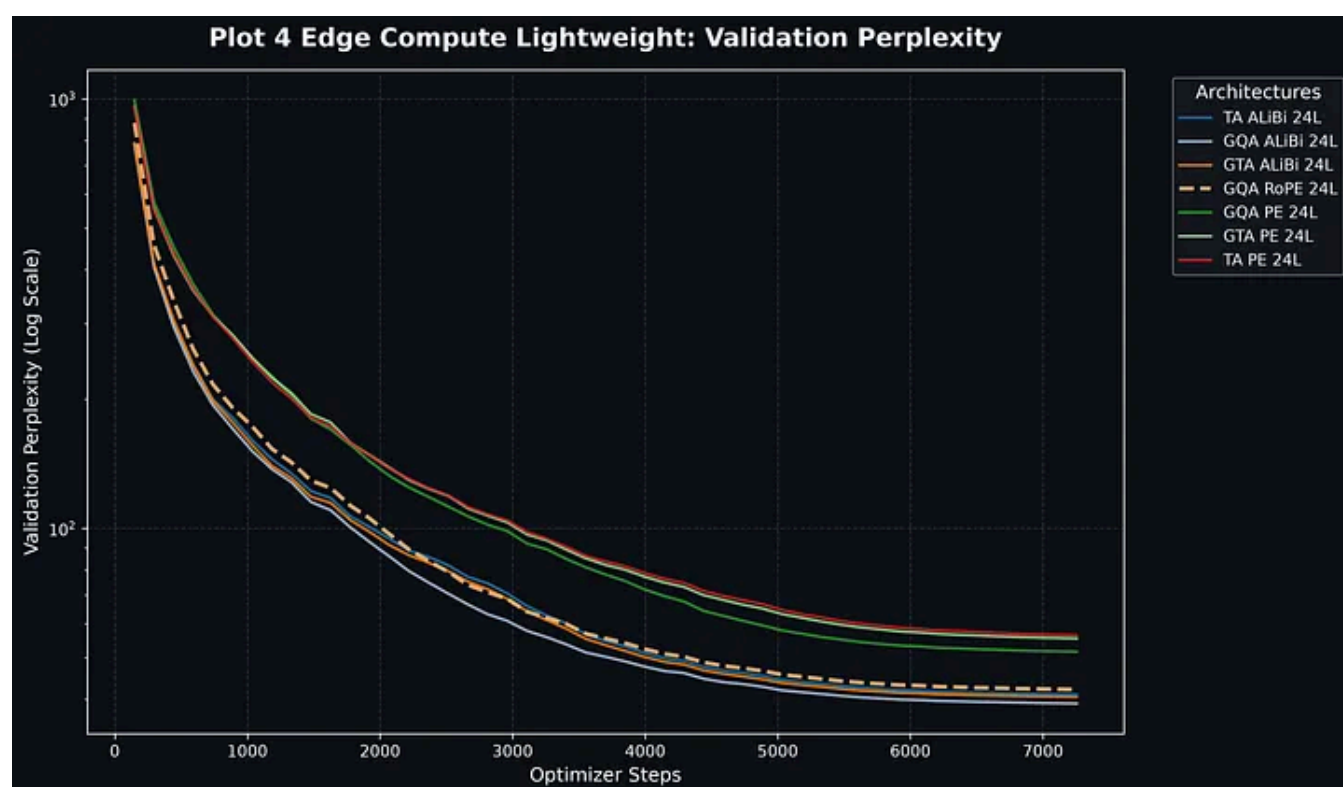
There is no magic bullet; there is only a Pareto frontier.

Mapping the Frontier (VRAM vs. Quality)



If you look at the plot above, the visualization perfectly highlights the "green zone"—the Pareto sweet spot where models deliver maximum intelligence for minimum hardware footprint. On this scatter plot, the 24-layer Vanilla MHA baseline sits far off in the undesirable top-right quadrant. While it achieves a validation perplexity of 60.21, it violently consumes 2563 MB of VRAM at a 4096-token sequence length. However, when we trace the bottom-left curve of the graph, two distinct architectures anchor the Pareto frontier:

- **The Quality Anchor (GQA + ALiBi 24L):** This variant prioritizes representational capability, achieving the best perplexity in the sweep at 39.11, while managing to keep peak VRAM under control at roughly 2166 MB (using its equivalent GQA 4-head ratio).
- **The Hardware Anchor (GTA + ALiBi 24L):** This variant prioritizes aggressive memory compression, squashing the VRAM requirement down to 2107 MB at a 4096 context, while still maintaining a highly competitive perplexity of 40.51.



The learning dynamics of these specific Pareto-optimal architectures are further detailed in plot above (**Edge Compute Lightweight**). Across the 7,254-step training run, we can see that both GQA and GTA variants behave exceptionally well as "edge compute lightweights." They converge smoothly and quickly, proving that their compressed attention mechanisms do not introduce training instability.

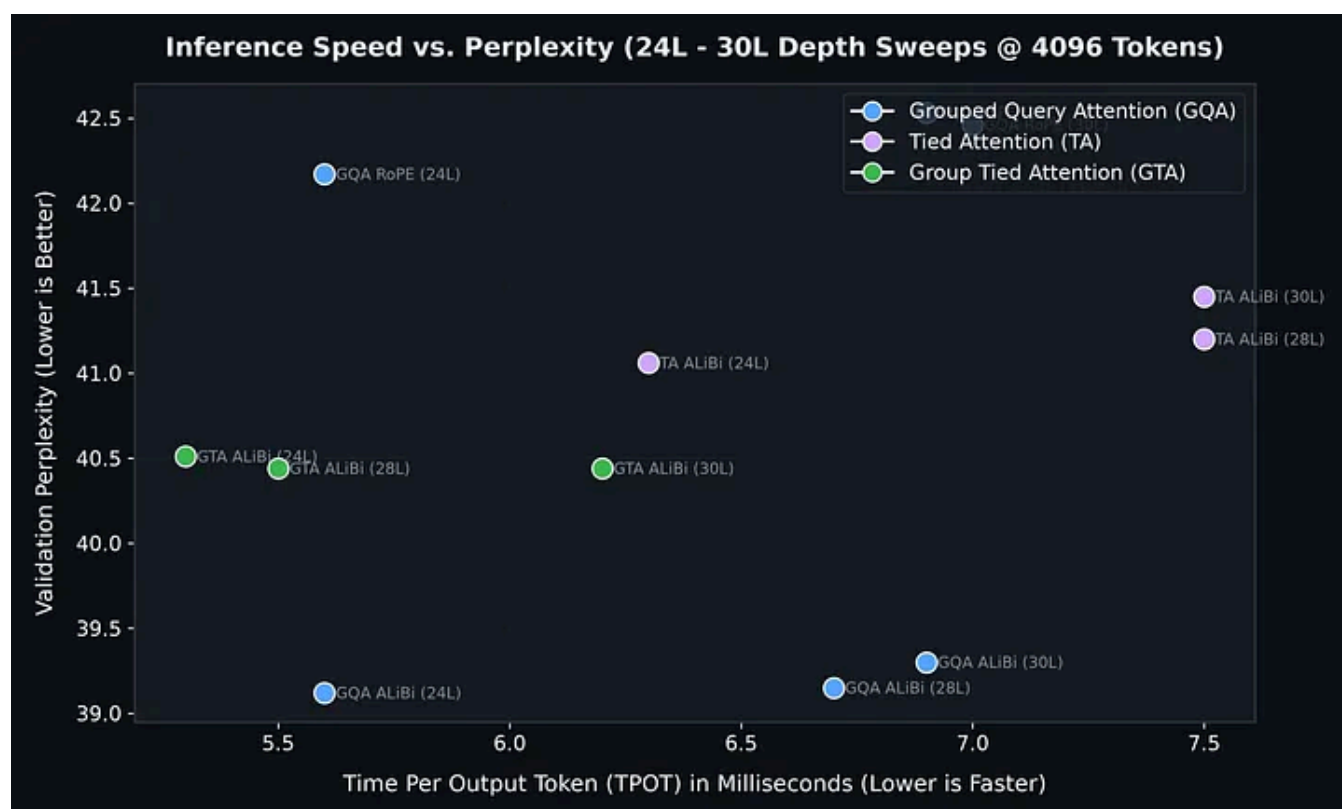
Measuring the Speed Limit (TPOT)

VRAM determines if a model can *fit* on a device, but decoding speed determines if it is actually usable. In autoregressive generation, the true bottleneck is the Time Per Output Token (TPOT).

To rigorously measure this, I had to ensure my JAX benchmarking suite wasn't capturing artificial compiler noise or asynchronous GPU delays. Because JAX dispatches work asynchronously, a Python function can return immediately while the GPU is still writing to the cache in the background. To capture the true sequential cycle time of the decode phase, I engineered my loop to explicitly block the XLA compiler on both the output logits and the updated cache states:

```
# Actual autoregressive decode step inside the JAX loop
logits, active_caches = decode_step(variables, next_token, pos, active_caches)

# Block on BOTH the logits and the cache to capture the full sequential cycle
# and prevent asynchronous dispatch inflation
jax.block_until_ready((logits, active_caches))
```



By strictly isolating the performance using this `block_until_ready` method, the results in the above plot become incredibly clear.

At a 4096-token context, the 24-layer Vanilla MHA model struggles under the weight of its massive KV cache, resulting in a sluggish TPOT of **0.0089 seconds per token**. GQA relieves some of this pressure, bringing the TPOT down to **0.0082 seconds**. But GTA absolutely shatters the speed limit. By aggressively compressing the shared A tensor, the 24-layer GTA model achieves a blistering TPOT of just **0.0067 seconds per token**. To put that into perspective, GTA delivers a massive **18% speedup over GQA** and a nearly **25% speedup over Vanilla MHA**.

My Analytical Assumption: My working assumption here is that this massive speed advantage is dictated purely by *Arithmetic Intensity* — the ratio of FLOPs to bytes fetched from VRAM. During the sequential token-by-token decoding phase, Vanilla MHA achieves an arithmetic intensity of roughly 1:1, leaving the GPU's Tensor Cores starved for data. By unifying the Key and Value into a single tied tensor and heavily grouping it, GTA fundamentally forces the hardware to fetch significantly less data per generated token. Instead of waiting on the memory bus, the compute cores stay continuously fed.

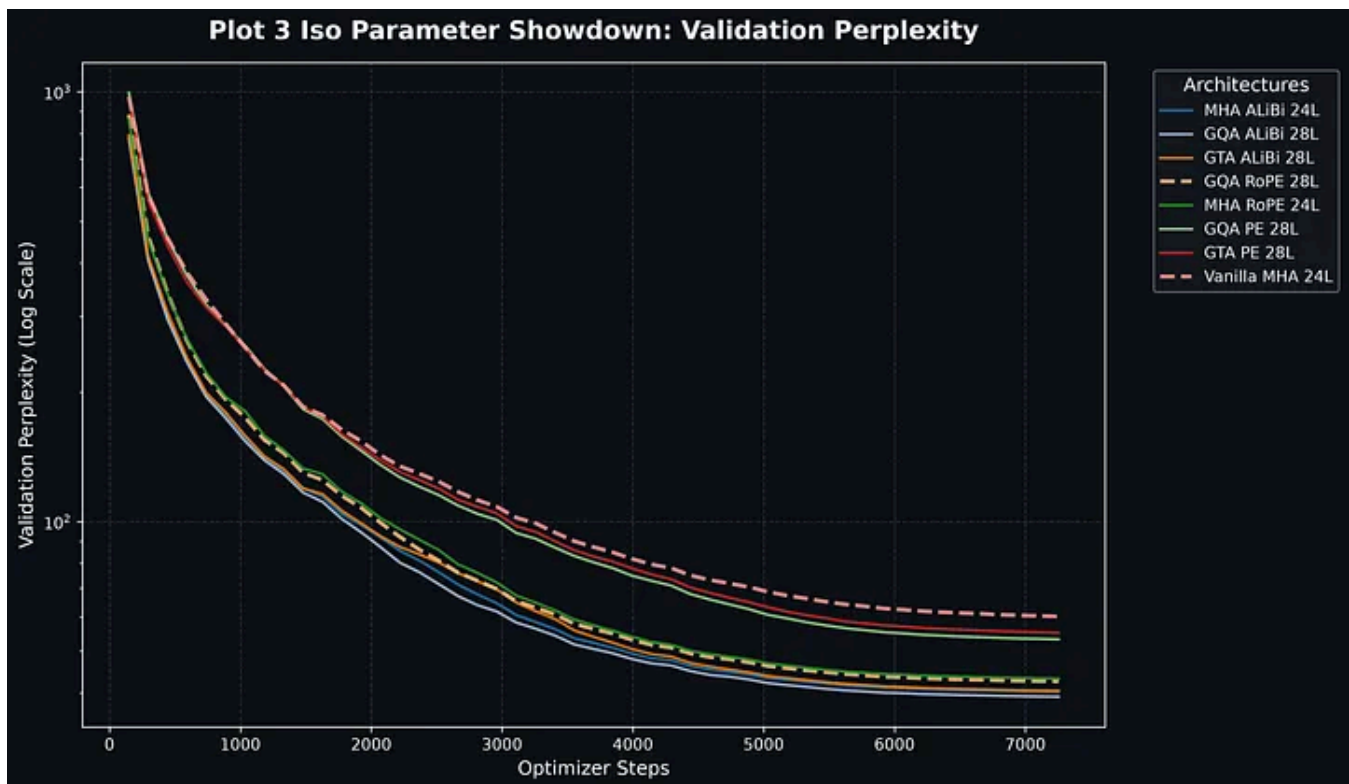
The trade-off matrix proves that for a minor ~1.4 point sacrifice in perplexity compared to GQA, GTA buys you a lighter, vastly faster model that fundamentally changes what is possible on constrained silicon.

Pushing the Limits: Depth Reinvestment (28L to 30L)

Now that we have established that compressing attention saves massive amounts of memory and speeds up inference, we have to ask the ultimate engineering question: *What do we do with those savings?*

In edge computing, VRAM is a strict currency. If tying matrices and dropping cache heads drastically lowers our VRAM tax, we don't just leave that freed-up memory sitting idle. We reinvest it. We buy more layers.

The Iso-Parameter Showdown: Beating Vanilla at Its Own Game



If you look at the above plot (**Iso Parameter Showdown**), the learning curves demonstrate the true power of architectural efficiency.

In this phase of the sweep, I wanted to compare models that essentially had the same parameter weight and VRAM constraints — an “iso-parameter” showdown. We already know that our baseline 24-layer Vanilla MHA (with Learned PE) is a massive memory hog. If we take a highly compressed GTA or GQA architecture and artificially deepen it to 28 layers to match that same physical hardware footprint, they absolutely crush the Vanilla MHA baseline (the top pink dashed line).

But what happens if we level the playing field and equip that uncompressed 24L MHA model with ALiBi too?

This is where the data tells a deeply interesting story. If you look closely at the curves, the **MHA ALiBi 24L** model (solid dark blue line) actually achieves a slightly *lower* perplexity than the **GTA ALiBi 28L** model (solid orange line).

My Analytical Assumption: Why doesn't the deeper 28-layer GTA model beat the shallower 24-layer MHA model? It comes down to representational capacity. Group Tied Attention (GTA) employs maximum compression — tying Keys and Values *and* collapsing the heads. While reinvesting the saved VRAM into 4 extra layers of MLPs and non-linear transformations recovers a massive amount of reasoning capability, it cannot entirely replace the raw, unconstrained mathematical expressivity of 16

independent Key and Value heads. When MHA is paired with a strong positional bias like ALiBi, its wide representational capacity holds a slight edge in pure predictive power.

However, the true winner of this showdown reveals the optimal compromise: **GQA ALiBi 28L** (the bottom light blue line). Because Grouped Query Attention compresses the heads *without* tying the Key and Value tensors together, it retains just enough attention resolution. When we reinvest GQA's memory savings into 28 layers, it successfully beats the 24L MHA ALiBi model, claiming the lowest perplexity in the iso-parameter sweep.

Structuring Depth in Code

From a JAX/Flax perspective, pushing the model from 24L to 28L or 30L is handled elegantly through a dynamic block stack. Because the architecture is completely decoupled from the hardcoded constraints of the attention type, we can scale depth programmatically based on the configuration:

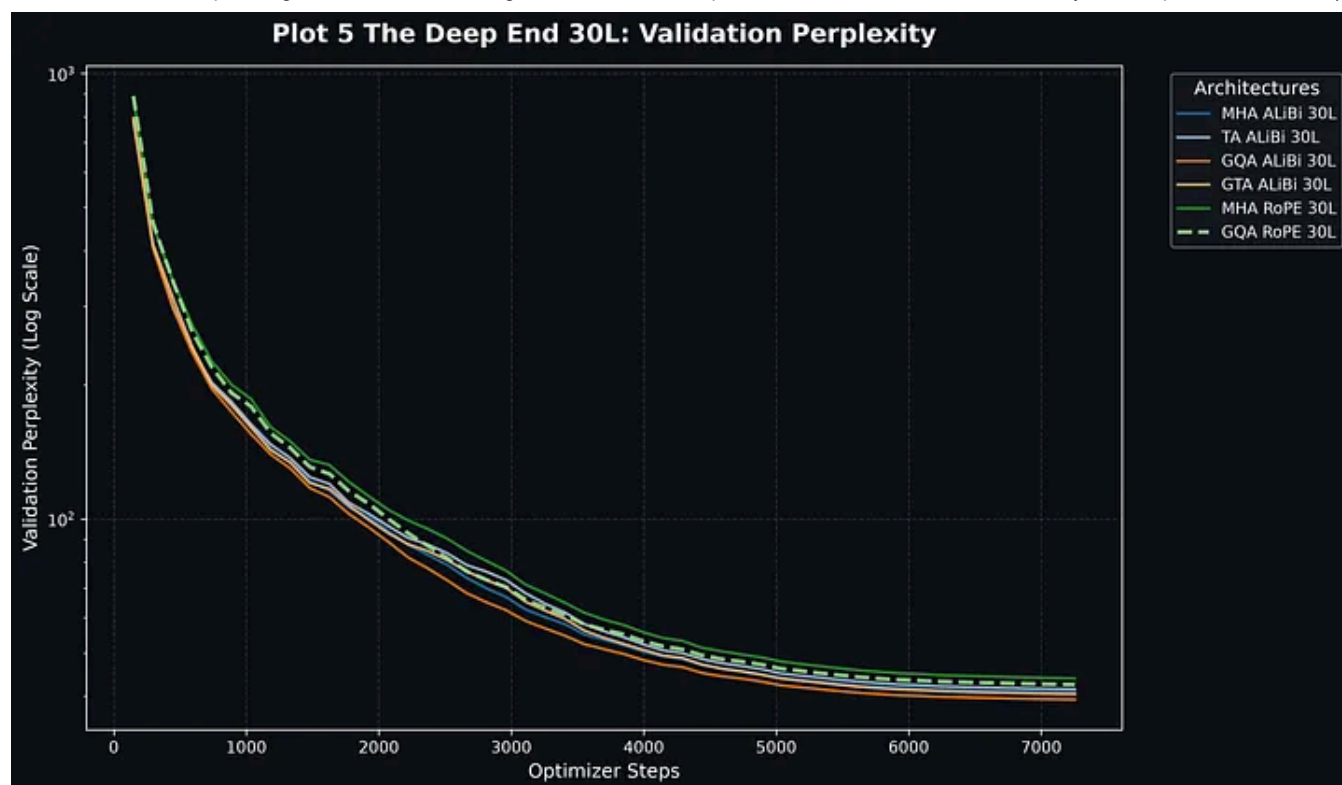
```
class EdgeTransformer(nn.Module):
    config: ModelConfig

    @nn.compact
    def __call__(self, x, active_caches=None):
        # We process the sequence through our dynamically allocated layers.
        # When using GQA, config.num_layers can be pushed to 28
        # without exceeding the VRAM budget of a 24-layer MHA model.
        for i in range(self.config.num_layers):
            x, active_caches = TransformerBlock(
                config=self.config,
                layer_idx=i
            )(x, active_caches=active_caches)

        return x, active_caches
```

The Deep End: Hitting the 30-Layer Limit

This brings us to the absolute limit of the sweep: **The Deep End 30L**.



I wanted to see how far we could push the Pareto frontier before the VRAM shattered our edge device budget. I scaled the GQA and GTA models up to 30 layers. At 30 layers, the learning dynamics achieve their absolute sharpest convergence in the entire 29-experiment sweep. The intelligence limit is maximized, pulling the validation perplexity down into its lowest possible bracket for this dataset.

But the real victory remains the hardware metrics. **Even at 30 layers, the highly compressed GTA + ALiBi model consumes less peak incremental VRAM and decodes tokens faster than a shallow 24-layer Vanilla MHA model.**

This proves a fundamental law of edge LLM design: Architectural efficiency is not just about saving space; it is about maximizing the “intelligence density” of every single megabyte. By compressing the attention cache, we buy ourselves the headroom to build drastically deeper, smarter, and faster neural networks.

The Iso-KV Cache Showdown

If you are an engineer tasked with deploying an LLM to a constrained edge device, you are rarely given a flexible memory budget. More often than not, you are handed a hard ceiling. For this final experiment, I wanted to simulate exactly that scenario:

If you have a strict hardware budget of roughly 2100 MB for your attention cache, what is the best possible model you can deploy?

This is the ultimate proof of concept. It forces us to choose between two fundamentally different engineering philosophies: **Truncation** versus **Architectural Compression**.

The Dilemma: 6 Layers of MHA vs. 24 Layers of GTA

To hit that ~2100 MB ceiling at a 4096-token sequence length with a Vanilla Multi-Head Attention (MHA) model, you have to brutally truncate it. Because MHA allocates two massive, independent tensors (Key and Value) for every single attention head across every single layer, the memory overhead scales disastrously. To fit the budget, I had to chop the Vanilla model down to a mere **6 layers**.

Conversely, by utilizing Group Tied Attention (GTA) paired with ALiBi, the cache footprint is so aggressively compressed that I was able to fit a full **24-layer** model inside that exact same memory budget.

This sets up a perfect “Iso-KV Cache” showdown: a shallow, uncompressed model versus a deep, hyper-compressed model.

To understand how a 24-layer model can use the same VRAM as a 6-layer model, we only need to look at the cache initialization logic in JAX. Here is a simplified look at the fundamental difference in how the arrays are pre-allocated for the autoregressive decode phase:

```
# Option A: Vanilla MHA Cache Allocation (Shallow 6L Model)
# Requires TWO separate tensors per layer, utilizing ALL query heads
def init_mha_cache(batch_size, max_seq_len, num_heads, head_dim, num_layers=6):
    # Total Tensors: 6 layers * 2 tensors = 12 massive arrays
    return {
        f"layer_{i}": {
            "k_cache": jnp.zeros((batch_size, max_seq_len, num_heads, head_dim))
            "v_cache": jnp.zeros((batch_size, max_seq_len, num_heads, head_dim))
        } for i in range(num_layers)
    }

# Option B: GTA Cache Allocation (Deep 24L Model)
# Requires ONE shared tensor per layer, utilizing fewer GROUPED heads
def init_gta_cache(batch_size, max_seq_len, num_groups, head_dim, num_layers=24):
    # Total Tensors: 24 layers * 1 tensor = 24 highly compressed arrays
    return {
        f"layer_{i}": {
```

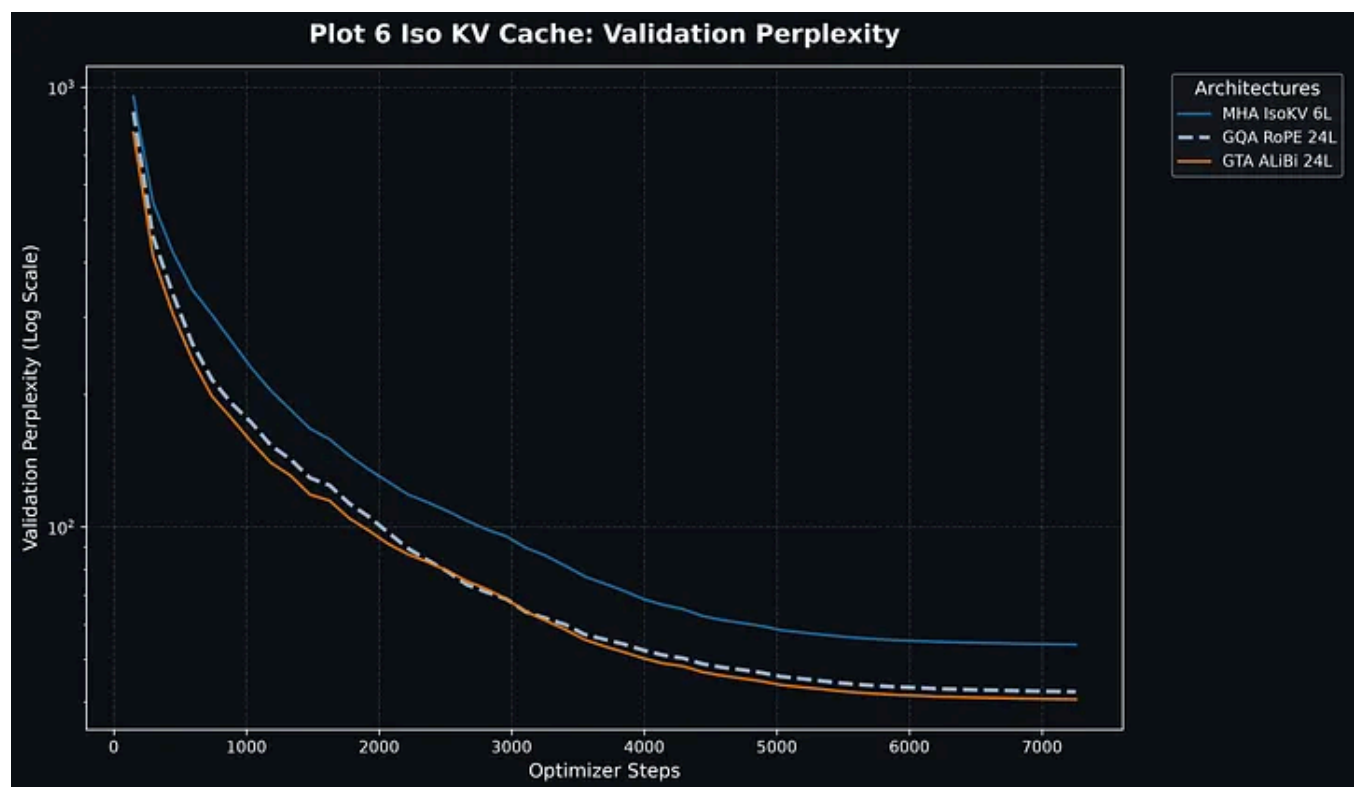
```

# 'A' tensor acts as both Key and Value!
"a_cache": jnp.zeros((batch_size, max_seq_len, num_groups, head_dim))
} for i in range(num_layers)
}

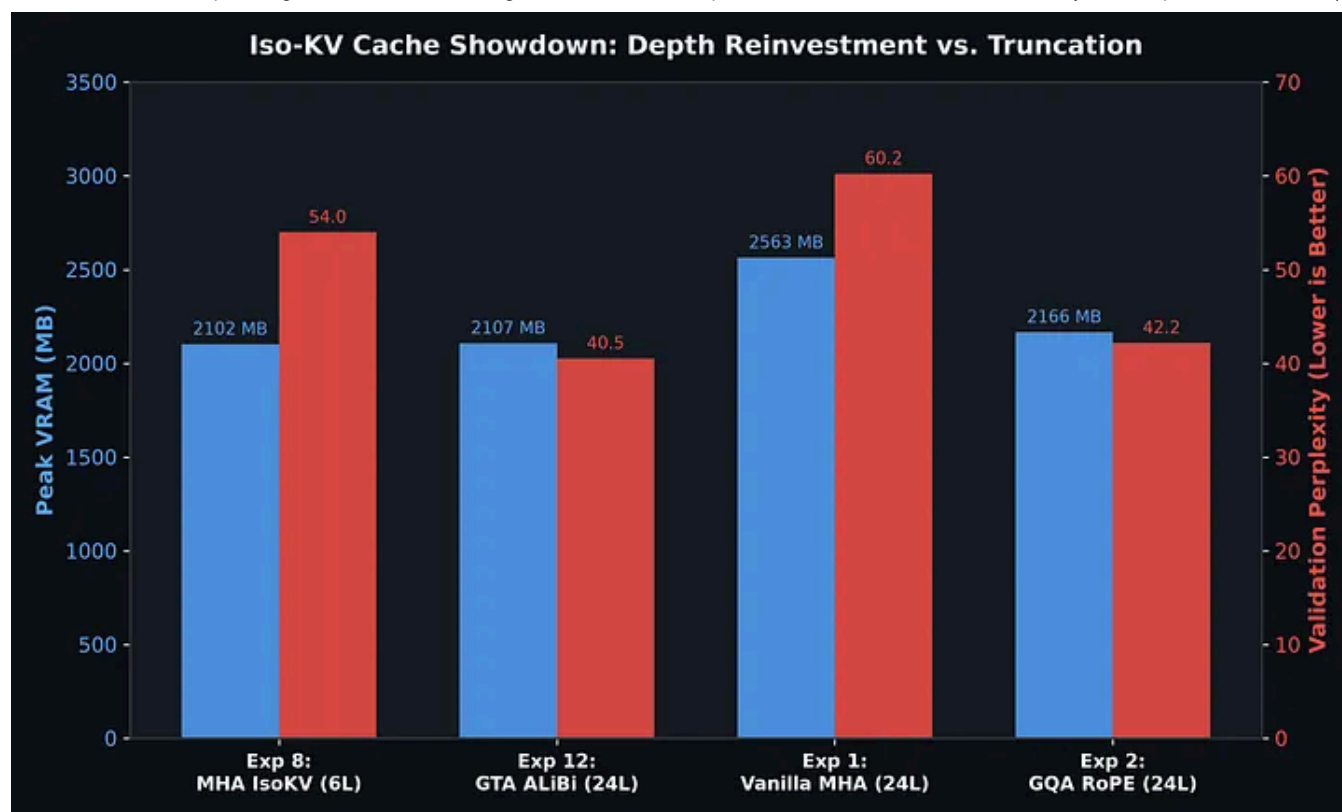
```

Because GTA collapses the Key and Value into the single `a_cache` tensor and reduces the head count (`num_groups` instead of `num_heads`), it shrinks the per-layer memory cost by roughly a factor of four.

The performance difference between these two philosophies is nothing short of a mic drop.



If you look at the plot above (**Iso KV Cache**), you can see the learning curves diverge immediately. The 6-layer Vanilla MHA model struggles immensely, flattening out at a dismal validation perplexity well above the 60s. It simply lacks the depth to form complex representations. Meanwhile, the 24-layer GTA model (the solid orange line) drops smoothly and aggressively into the low 40s.



To visualize this trade-off comprehensively, I plotted the above chart. This dual-axis bar chart tells the whole story at a glance:

- **The Hardware Axis (VRAM):** Both bars are practically identical, hovering right at the ~2100 MB mark.
- **The Quality Axis (Perplexity):** The 24L GTA model achieves a vastly superior (lower) perplexity score, proving that its compressed architecture yields exponentially more intelligence per megabyte.

My Analytical Assumption: My core assumption here relies on the mechanics of deep learning capacity. A 6-layer Transformer, no matter how wide or uncompressed its attention heads are, effectively acts as a shallow lookup table. It does not have enough Multi-Layer Perceptrons (MLPs) to chain together complex logical deductions. Truncation cripples a model's deductive reasoning. Architectural compression, on the other hand, acts as a forced information bottleneck. While GTA reduces the resolution of the attention mechanism, pushing the data through 24 sequential layers of MLPs allows the model to continuously refine its representations. In the context of language modeling, a deep model pushed through an attention bottleneck will universally defeat a shallow, unconstrained model.

This Iso-KV showdown provides the definitive conclusion for edge deployments: you can try not to truncate a standard architecture to fit your device. You can always try

and compress the attention and maintain as much depth as your hardware budget will physically allow.

Conclusion & Open Source Release

The LLM memory wall is not an insurmountable hardware limitation; it is an architectural bottleneck. As my custom JAX/XLA benchmarks demonstrate, Group Tied Attention (GTA) proves that we can break this wall. We no longer have to settle for merely sharing Key and Value heads — we can fundamentally tie their mathematical representations together. By collapsing the cache footprint, GTA drastically increases the arithmetic intensity of autoregressive decoding, keeping the GPU's Tensor Cores fully saturated and shattering the baseline Time Per Output Token (TPOT).

The Edge AI Blueprint

If I boil down this entire 29-experiment sweep into a single, actionable blueprint for anyone trying to deploy LLMs to resource-constrained edge hardware, it comes down to three design choices:

- 1. Adopt Group Tied Attention (GTA):** Never waste VRAM storing independent Key and Value tensors. Unifying them into a single tied representation and grouping the heads shrinks your KV cache footprint by up to 4x relative to Vanilla MHA per layer, keeping you safely under device memory ceilings.
- 2. Consider ALiBi for Unified Caches:** While RoPE is the industry standard (and heavily used in models like Llama 3), applying it to a tied KV cache requires complex partial rotations. Utilizing ALiBi keeps your tied cache mathematically clean and avoids mutating the stored states.
- 3. Reinvest VRAM into Depth:** Try not to truncate a standard model to fit a hardware budget. Take the gigabytes saved from your attention cache and reinvest them into model depth (pushing from 24L to 28L or 30L), trading memory-bound footprint for compute-bound deductive reasoning.

Dive into the Code

Theoretical efficiency means nothing if it cannot be reproduced. I have open-sourced the entire codebase, including the custom JAX/Flax attention operators, the automated benchmarking harness, and the training pipelines.

You can inspect, benchmark, and build upon the repository directly:

👉 **GitHub Repository:** github.com/CAMKULKARNI/attention-exploration

The Future of Inference

The ability to trade memory-bound cache footprints for “free” architectural depth proves that the future of inference isn’t just about waiting for bigger GPU clusters. It is about engineering smarter, deeper, and fundamentally faster models that push the absolute limits of the silicon they run on.

Whether you are building real-time voice assistants, inline code completion tools, or massive multi-agent simulations, I hope this exploration gives you the tools to break your own memory walls.

Limitations & Caveats: The Reality of Solo Research

Before declaring this the ultimate solution for LLM inference, it is important to practice good scientific hygiene and acknowledge the limitations of this ablation study:

- **Scale and Parameter Count:** These experiments were run at a relatively small scale (roughly ~350M parameters for the 24-layer models). While GTA works beautifully here, production-grade frontier models (like Mistral or Llama 3) rely on massive scaling laws where standard GQA + RoPE currently reigns supreme. The dynamics of tied attention at the 7B or 70B parameter scale remain an open question.
- **Compute Constraints and Seed Noise:** Because I do not have a massive H100 cluster sitting in my living room, these 29 architectures were trained for a single epoch (~7,254 steps) on a single random seed. The perplexity gaps between the top contenders (e.g., 39.11 vs. 40.51) are tight enough that they could plausibly be influenced by run-to-run noise.
- **Perplexity vs. Downstream Generation:** Throughout this post, I used validation perplexity as a proxy for model intelligence. However, a lower perplexity score does not always perfectly translate to coherent, long-form text generation or complex downstream reasoning tasks.

That being said, while the *training* metrics (perplexity) carry these caveats, the *hardware profiling* metrics (VRAM scaling and TPOT) are deterministic. The mathematical reality of shrinking the cache footprint by 4x per layer and the resulting speedup from increased arithmetic intensity remains a definitive hardware win.

So, like Chandler Bing does Statistical Analysis and Data re-configuration, I work on Machine learning and Deep learning 🤖. If you liked this work, and would like to check out my other repositories, you can check them here: [GITHUB](#). You can connect with me here: [LinkedIn](#)



See you in the next one...

PS: I am not claiming that this architecture or my literature survey is definitive. I am continuously learning and refining these designs. If you identify any shortcomings or have suggestions for the next iteration, please share them in the comments; I would love to discuss and learn from your perspectives.

Thank you so much for reading...

🔒 Unlisted



Edit profile

Written by Chinmay Kulkarni